ANÁLISE #44 · MODO: PASSIVA

MT-01 Clique duplo 193241

http://127.0.0.1:8765/ · 2026-10-28 22:33:07 UTC

38
Pontuação de Segurança

Verificações executadas	Achados
34	12

Status HTTP	Duração
200	5047 ms

O que sua equipe deve corrigir

Todos 12 | ALTO 1 | MÉDIO 5 | BAIXO 6

ALTO Alta Confiança · 80% de confiança

Conexão não é criptografada

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

A URL informada usa o esquema http://.

Por que isso importa para sua aplicação

Qualquer pessoa no caminho de rede pode ler ou modificar requisições e respostas, incluindo credenciais e cookies de sessão.

Como corrigir

Redirecione toda requisição HTTP para HTTPS e configure um certificado TLS válido.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Site servido por http:// simples, sem
redirecionamento.
```

✅ Como corrigir Apache (.htaccess)

```
# Forca HTTPS para todo o trafego:
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^ https://%{HTTP_HOST}%
{REQUEST_URI} [R=301,L]
```

💡 Redirecione todo HTTP para HTTPS e instale um certificado TLS válido (ex.: Let's Encrypt).

▶ Contexto técnico

MÉDIO Alta Confiança · 78% de confiança

Cookie não possui política explícita entre sites

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Ao menos um cabeçalho Set-Cookie não possui SameSite.

Por que isso importa para sua aplicação

O navegador pode incluir o cookie em requisições entre sites, aumentando a exposição a CSRF.

Como corrigir

Defina `SameSite=Lax` ou `Strict`; use `None; Secure` somente quando o uso entre sites for necessário.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
setcookie('sessao', $id, ['secure' => true,
'httponly' => true]);
```

✅ Como corrigir PHP

```
setcookie('sessao', $id, ['secure' => true,
'httponly' => true, 'samesite' => 'Lax']);
```

💡 SameSite=Lax (ou Strict) reduz a superfície de CSRF.

▶ Contexto técnico

MÉDIO Alta Confiança · 78% de confiança

JavaScript pode ler um cookie

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Ao menos um cabeçalho Set-Cookie não possui HttpOnly.

Por que isso importa para sua aplicação

Se existir uma falha de XSS em outro ponto, scripts podem roubar cookies que não possuem HttpOnly.

Como corrigir

Defina `HttpOnly` em cookies que não precisam de acesso via JavaScript.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
setcookie('sessao', $id, ['secure' => true]);
// acessivel via JS
```

✅ Como corrigir PHP

```
setcookie('sessao', $id, ['secure' => true,
'httponly' => true, 'samesite' => 'Lax']);
```

💡 HttpOnly impede que scripts (XSS) leiam o cookie de sessão.

▶ Contexto técnico

MÉDIO

Alta Confiança · 76% de confiança

Navegador não possui política de execução de conteúdo

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Nenhum cabeçalho Content-Security-Policy foi retornado.

Por que isso importa para sua aplicação

Scripts injetados têm menos restrições em nível de navegador, aumentando o impacto de qualquer falha de XSS.

Como corrigir

Comece com uma Content-Security-Policy em modo somente-relatório, inventarie as origens necessárias e depois aplique uma política restritiva sem `unsafe-inline`.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Nenhuma Content-Security-Policy e enviada.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# Comece em modo relatorio, inventarie as
origens e depois aplique:
Header always set Content-Security-Policy
"default-src 'self'; object-src 'none'; base-
uri 'self'"
```

💡 Implante primeiro como Report-Only, ajuste as origens e então aplique sem unsafe-inline.

▶ Contexto técnico

MÉDIO Alta Confiança · 76% de confiança

Página pode ser incorporada em outro site

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Nem CSP frame-ancestors nem X-Frame-Options foram retornados.

Por que isso importa para sua aplicação

Um atacante pode sobrepor esta página e induzir usuários a clicar em controles sensíveis (clickjacking).

Como corrigir

Adicione `frame-ancestors 'none'` (ou origens confiáveis) ao Content-Security-Policy. X-Frame-Options: DENY é um fallback legado.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set Content-Security-Policy
"frame-ancestors 'none'"

// Alternativa em PHP, antes de qualquer saida:
header('Content-Security-Policy "frame-
ancestors 'none'"');
```

💡 frame-ancestors no CSP é o mecanismo atual; X-Frame-Options: DENY é fallback legado.

▶ Contexto técnico

MÉDIO

Alta Confiança · 75% de confiança

Resposta potencialmente vinculada à sessão sem política no-store

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

A resposta define cookies (ou varia por Authorization), mas Cache-Control não contém no-store.

Por que isso importa para sua aplicação

Navegadores compartilhados, proxies ou CDNs podem reter uma resposta associada à sessão de um usuário específico.

Como corrigir

Para respostas sensíveis e autenticadas, retorne `Cache-Control: no-store, private`.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
// Pagina autenticada sem diretiva de cache (pode ser guardada).
```

✅ Como corrigir PHP

```
header('Cache-Control: no-store, no-cache, must-revalidate, private');
header('Pragma: no-cache');
```

💡 Em respostas com dados sensíveis, use Cache-Control: no-store.

▶ Contexto técnico

BAIXO

Alta Confiança · 76% de confiança

Capacidades do navegador não são explicitamente restringidas

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Permissions-Policy está ausente.

Por que isso importa para sua aplicação

Conteúdo incorporado ou comprometido pode solicitar capacidades do navegador que a aplicação não precisa.

Como corrigir

Adicione uma Permissions-Policy que desabilite recursos não utilizados, por exemplo `camera=(), microphone=(), geolocation=()`.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set Permissions-Policy
"geolocation=(), camera=(), microphone=()"

// Alternativa em PHP, antes de qualquer saida:
header('Permissions-Policy "geolocation=(),
camera=(), microphone=()"');
```

💡 Desabilite recursos do navegador que a aplicação não usa.

▶ Contexto técnico

BAIXO Alta Confiança · 76% de confiança

Detalhes do servidor web são expostos

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

O cabeçalho de resposta Server expõe informações de produto ou versão.

Por que isso importa para sua aplicação

Indícios de versão ajudam atacantes a priorizar técnicas específicas do produto e CVEs conhecidas.

Como corrigir

Remova detalhes de versão do cabeçalho Server no servidor web ou proxy reverso.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
ServerTokens Full          # expoe versao exata
do servidor
```

✅ Como corrigir Apache

```
ServerTokens Prod
ServerSignature Off
```

💡 Reduza a informação de versão nos cabeçalhos do servidor.

▶ Contexto técnico

BAIXO Alta Confiança · 76% de confiança

Janela de nível superior não está isolada

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Cross-Origin-Opener-Policy está ausente.

Por que isso importa para sua aplicação

Sem COOP, janelas de origem cruzada podem manter referências que habilitam ataques de canal lateral ou baseados em opener.

Como corrigir

Adicione `Cross-Origin-Opener-Policy: same-origin` após testar integrações e pop-ups de login.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set Cross-Origin-Opener-Policy
"same-origin"

// Alternativa em PHP, antes de qualquer saida:
header('Cross-Origin-Opener-Policy "same-
origin"');
```

💡 Isola o contexto de navegação de janelas de outras origens.

▶ Contexto técnico

BAIXO

Alta Confiança · 76% de confiança

Navegador pode adivinhar o tipo de conteúdo da resposta

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

X-Content-Type-Options está ausente ou não é `nosniff`.

Por que isso importa para sua aplicação

Conteúdo enviado ou servido com o tipo errado pode ser interpretado como conteúdo executável pelo navegador.

Como corrigir

Retorne o Content-Type correto e adicione `X-Content-Type-Options: nosniff`.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set X-Content-Type-Options
"nosniff"

// Alternativa em PHP, antes de qualquer saida:
header('X-Content-Type-Options "nosniff"');
```

💡 Impede o navegador de adivinhar (sniff) o tipo de conteúdo.

▶ Contexto técnico

BAIXO Alta Confiança · 76% de confiança

Política de carregamento de recursos não é explícita

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Cross-Origin-Resource-Policy está ausente.

Por que isso importa para sua aplicação

Outras origens podem conseguir incorporar este recurso, a menos que outro comportamento da aplicação impeça isso.

Como corrigir

Para recursos não públicos, defina `Cross-Origin-Resource-Policy: same-origin` ou `same-site`.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set Cross-Origin-Resource-Policy
"same-origin"

// Alternativa em PHP, antes de qualquer saida:
header('Cross-Origin-Resource-Policy "same-
origin"');
```

💡 Impede que outras origens embutam este recurso.

▶ Contexto técnico

BAIXO Alta Confiança · 76% de confiança

URLs podem vazar pelo cabeçalho Referer

📍 `GET http://127.0.0.1:8765/`

O que o scanner observou

Referrer-Policy está ausente.

Por que isso importa para sua aplicação

Caminhos e parâmetros de query podem ser expostos a sites de terceiros quando usuários seguem links externos.

Como corrigir

Adicione `Referrer-Policy: strict-origin-when-cross-origin` e nunca coloque segredos em URLs.

EXEMPLO PRÁTICO

❌ Padrão vulnerável

```
# Resposta atual: o cabecalho abaixo NAO e
enviado.
```

✅ Como corrigir Apache (.htaccess) / PHP

```
# .htaccess (ou vhost) — modulo headers
habilitado:
Header always set Referrer-Policy "strict-
origin-when-cross-origin"

// Alternativa em PHP, antes de qualquer saida:
header('Referrer-Policy "strict-origin-when-
cross-origin"');
```

💡 Evita vazar a URL completa (com tokens) para terceiros.

▶ Contexto técnico

Escopo e limitações

Esta é uma avaliação passiva: apenas observação de respostas, sem envio de payloads de teste. Ela não prova a ausência de vulnerabilidades e não substitui um teste de intrusão profissional.